Лабораторная работа 4 Регистрация пользователя и модели разграничения доступа

Задание выполняется в трех вариантах. Общий стек и базовая регистрация одинаковы для всех вариантов, отличается только модель управления доступом.

1 Цель работы

Разработать минимальное веб-приложение, в котором пользователь проходит регистрацию через форму на frontend, backend создает запись пользователя в базе данных, пароль сохраняется только в виде хэша, а интерфейс получает понятный статус выполнения операции.

После реализации базовой регистрации команда дополняет проект имитацией одной модели разграничения доступа по назначенному варианту: мандатной, дискреционной или ролевой.

2 Общий стек

- Frontend: React с Vite или Create React App, желательно TypeScript.
- Backend: Python, предпочтительно FastAPI; допускается Flask при сохранении тех же требований к API.
- База данных: PostgreSQL.
- ORM и миграции: SQLAlchemy или SQLModel, миграции через Alembic.
- Контейнеризация: отдельные Dockerfile для frontend и backend, docker-compose.yml поднимает frontend, backend и db.
- Конфигурация: .env и .env.example с переменными DATABASE_URL, HASH_SCHEME, SECRET_KEY, APP_ENV, PORT и параметрами хэширования.

3 Базовый функционал для всех вариантов

Регистрационная форма

- На странице /register должна быть форма с полями login и password.
- Форма отправляет запрос POST /api/register на backend.
- При успешной регистрации backend возвращает JSON-ответ, например {"message": "user создан"}.
- На UI нужно показать успешный результат или понятное сообщение об ошибке.

Создание пользователя

- В базе данных создается таблица users.
- Минимальные поля таблицы: id, login, password_hash, created_at.
- Поле login должно быть уникальным.
- Пароль не должен сохраняться и не должен логироваться в открытом виде.

Валидация

- login: длина от 3 до 32 символов, разрешены латинские буквы, цифры, точка, нижнее подчеркивание и дефис.
- password: минимум 8 символов, минимум одна заглавная буква, одна строчная буква, одна цифра и один специальный символ.

- Ошибки должны возвращаться с понятным текстом: 422 для ошибок валидации, 409 для уже занятого логина.

4 Варианты задания

Каждая команда реализует только один вариант. Базовая регистрация, стек, Docker, база данных, логирование, тесты и README остаются обязательными для всех.

Вариант	Модель	Что реализовать	Минимальная проверка
1	Мандатная модель доступа	У каждого пользователя есть уровень допуска. У каждого защищаемого ресурса есть уровень секретности. Доступ разрешается только по правилам сравнения уровней.	Показать ресурс, доступный пользователю, и ресурс, который пользователь не может открыть из-за недостаточного допуска.
2	Дискреционная модель доступа	У каждого ресурса есть владелец. Владелец может выдавать другим пользователям права на чтение, изменение или удаление ресурса.	Показать, что один пользователь выдал право другому пользователю, а пользователь без права получил отказ.
3	Ролевая модель доступа	Права назначаются не напрямую пользователю, а ролям. Пользователь получает одну или несколько ролей, а доступ определяется набором прав этих ролей.	Показать разные возможности для ролей user, moderator и admin или для аналогичного набора ролей.

5 Требования к реализации моделей доступа

Вариант 1 Мандатная модель доступа

- Добавить пользователям уровень допуска, например public, internal, confidential, secret.
- Добавить защищаемые ресурсы с уровнем секретности.
- Реализовать проверку доступа на backend. Пользователь не должен получать ресурс, если его уровень допуска ниже уровня ресурса.
- На frontend показать список ресурсов и результат проверки доступа для текущего пользователя.
- В README описать выбранные уровни и правило, по которому backend принимает решение.

Вариант 2 Дискреционная модель доступа

- Добавить сущность ресурса, у которой есть владелец.
- Добавить таблицу прав доступа, например resource_permissions: user_id, resource_id, can_read, can_update, can_delete.

- Реализовать выдачу прав владельцем ресурса другому пользователю.
- Проверять права на backend перед чтением, изменением или удалением ресурса.
- На frontend показать сценарий: владелец выдает право, пользователь с правом получает доступ, пользователь без права получает отказ.

Вариант 3 Ролевая модель доступа

- Добавить роли, например user, moderator, admin.
- Добавить связь пользователей с ролями и связь ролей с правами.
- Реализовать проверку прав на backend на основе роли пользователя.
- На frontend показать различия между ролями: обычный пользователь видит только доступные действия, moderator получает расширенные права, admin получает полный доступ.
- В README описать роли, права каждой роли и примеры запросов.

6 Безопасность

- Использовать Argon2id или bcrypt для хэширования паролей. Рекомендуется Argon2id через argon2-cffi или passlib[argon2].
- Использовать соль и настраиваемые параметры хэширования. Для Argon2id вынести ARGON2_TIME_COST, ARGON2_MEMORY_COST и ARGON2_PARALLELISM в конфигурацию.
- Не хранить пароль в открытом виде и не выводить его в логи.
- Защитить регистрацию от дублирования логина через уникальный индекс.
- Проверку модели доступа выполнять на backend. Ограничения только на frontend не засчитываются.
- В README добавить раздел "Безопасность: что сделано".

7 Логирование и тесты

- Backend должен вести структурированные логи с уровнями INFO и ERROR.
- Нужно логировать успешную регистрацию, попытку регистрации с занятым логином и ошибки валидации без вывода сырого пароля.
- Минимум 3 автотеста на pytest: успешная регистрация, дублирующий login, слабый password.
- Дополнительно минимум 2 теста на выбранную модель доступа: разрешенный доступ и запрещенный доступ.

8 Что сдать

- Ссылку на репозиторий или архив проекта.
- README с инструкцией запуска через docker compose up, структурой проекта, описанием API и curl-примерами.
- Скриншоты работы формы регистрации и демонстрации выбранной модели доступа.
- Скриншот ветки Git и краткое описание вклада каждого участника команды: backend, frontend, database, тесты и документация.
- Миграции базы данных и .env.example без реальных секретов.

9 Критерии оценки

- Регистрационная форма работает и создает пользователя в PostgreSQL.
- Пароль сохраняется только в виде корректного хэша.
- Выбранная модель доступа реализована на backend и демонстрируется через UI или API.
- Проект запускается одной командой docker compose up.
- README позволяет повторить запуск и проверить основные сценарии без устных пояснений.
- Тесты запускаются и покрывают регистрацию и выбранную модель доступа.